

Курс лекций: «Введение в разработку программ»

Лекция. Создание консольных программ на C#

Словарик к лекции

Термин	Что означает
Консольное приложение	Программа без графического интерфейса: общение с пользователем через текст — окно с чёрным фоном и белым текстом (или наоборот)
CLI (<i>Command Line Interface</i>)	Интерфейс командной строки: способ управления программой через ввод текстовых команд
Поток ввода/вывода	Каналы, через которые программа получает данные (stdin — клавиатура) и отдаёт результаты (stdout — экран)
Точка входа	Метод, с которого начинается выполнение программы — <code>Main</code>
Буфер	Промежуточное хранилище данных при вводе-выводе
Сборка проекта (<i>build</i>)	Превращение исходного кода в готовую к запуску сборку (.exe/.dll)

5.1. Что такое консольное приложение и зачем оно будущему разработчику

 **Консольное приложение** — программа, взаимодействующая с пользователем исключительно текстом: она печатает сообщения на экран (в окно терминала/консоли)

и читает введённые с клавиатуры строки. У неё нет кнопок, окон и графики.

💡 **Аналогия:** если программа с графическим интерфейсом — это ресторан с официантами и красивым интерьером, то консольная программа — это окошко выдачи: просто и по делу. Вы говорите — она отвечает.

Почему обучение начинается с консоли:

1. **Ничего лишнего** — вы учитесь именно программировать (логика, данные, алгоритмы), а не рисовать интерфейсы.
2. **Все профессионалы используют консоль** — инструменты разработчика (dotnet , git , серверы) — консольные программы.
3. **Бэкэнд начинается с консоли** — веб-сервер ASP.NET Core внутри запускается... как консольное приложение.
4. **Минимум магии** — весь код на экране, ничего не скрыто в визуальном дизайнере.

5.2. Создаём первый проект: два способа

Способ 1. Командная строка (профессиональный)

Откройте терминал (PowerShell, cmd или терминал в VS Code) и выполните:

```
dotnet new console -n HelloApp    # создать проект-консоль с именем HelloApp
cd HelloApp                       # перейти в папку проекта
dotnet run                        # скомпилировать и запустить
```

Разбор команд:

Команда	Что делает
dotnet new console -n Имя	Создаёт шаблон консольного проекта
dotnet run	Компилирует (при изменениях) и запускает программу
dotnet build	Только компилирует, не запуская (проверка ошибок)
dotnet publish -c Release	Создаёт готовую к распространению сборку

Способ 2. Visual Studio / VS Code

- **Visual Studio:** File → New → Project → **Console App** → имя проекта → Create.
- **VS Code:** установите расширение **C# Dev Kit**, затем в терминале VS Code — те же команды `dotnet new console`.

Что лежит в папке проекта

```

HelloApp/
├─ HelloApp.csproj    ← «паспорт» проекта: целевая платформа, зависимости
├─ Program.cs         ← ВАШ КОД (точка входа)
├─ bin/               ← результат компиляции (готовые .dll/.exe)
└─ obj/               ← временные файлы компилятора (не трогаем)

```

Содержимое `HelloApp.csproj`:

```

<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>      <!-- Exe = исполняемая консольная программа -->
    <TargetFramework>net8.0</TargetFramework>  <!-- целевая платформа -->
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>

```

5.3. Анатомия первой программы

Откройте `Program.cs`. Вы увидите:

```

// Вариант А — современный (top-level statements, C# 9+):
Console.WriteLine("Hello, World!");

// Вариант В — классический вид (то же самое, «под капотом» А):
using System;                                // подключение пространств имён

namespace HelloApp                            // пространство имён — «адрес» ваших классов
{
    class Program
    {
        static void Main(string[] args)    // ТОЧКА ВХОДА: отсюда начинается выполнение
        {
            Console.WriteLine("Hello, World!");
        }
    }
}

```

```
}  
  
}
```

Разбор каждого элемента:

Элемент	Что это
<code>using System;</code>	Подключение базовой библиотеки (там живёт класс <code>Console</code>)
<code>namespace</code>	Пространство имён — «контейнер» для классов, чтобы имена из разных библиотек не конфликтовали
<code>class Program</code>	Главный класс программы
<code>static void Main(string[] args)</code>	Точка входа. <code>static</code> — вызывается без создания объекта; <code>void</code> — ничего не возвращает; <code>args</code> — аргументы командной строки (см. 5.9)
<code>Console.WriteLine(...)</code>	Команда «напечатать строку на экран и перейти на новую строку»

 **Почему современный вариант короче:** компилятор сам генерирует `class Program` и `Main` — достаточно написать тело программы. Дальше в лекциях используем краткий стиль.

5.4. Вывод на экран: `Console.Write` и `Console.WriteLine`

Класс `Console` — «пульт связи с консолью». Два главных метода вывода:

Метод	Поведение
<code>Console.WriteLine(x)</code>	Напечатать <code>x</code> и перейти на новую строку
<code>Console.Write(x)</code>	Напечатать <code>x</code> и остаться на той же строке

```
Console.WriteLine("Первая строка");  
Console.WriteLine("Вторая строка");  
Console.Write("Без");  
Console.Write("переноса");  
Console.WriteLine("!"); // напечатает: Безпереноса!
```

```
// Вывод чисел и выражений — автоматически превращаются в текст:  
Console.WriteLine(2 + 2);           // 4 — сначала вычисляется, потом печатается  
Console.WriteLine("2 + 2");         // 2 + 2 — в кавычках ТЕКСТ, не вычисляется  
Console.WriteLine(10 / 3);          // 3 — целочисленное деление (см. лекцию 6)  
Console.WriteLine(10 / 3.0);        // 3.3333333333333335 — дробное деление
```

Форматированный вывод: три способа подставить значение в текст

```
string name = "Анна";  
int age = 20;  
  
// Способ 1 — конкатенация (склеивание) знаком +. Работает, но громоздко:  
Console.WriteLine("Привет, " + name + "! Тебе " + age + " лет.");  
  
// Способ 2 — ИНТЕРПОЛЯЦИЯ строк (главный способ, $ перед кавычками):  
Console.WriteLine($"Привет, {name}! Тебе {age} лет.");  
  
// Внутри { } можно любое выражение:  
Console.WriteLine($"Через год будет {age + 1}");  
  
// Способ 3 — составное форматирование (старый стиль, встречается в чужом коде):  
Console.WriteLine("Привет, {0}! Тебе {1} лет.", name, age);
```

Красивый вывод чисел: спецификаторы формата

```
double price = 1234.5678;  
  
Console.WriteLine($"{price:F2}");    // 1234,57 — F2 = дробная часть, 2 знака  
Console.WriteLine($"{price:C}");     // 1 234,57 ₺ — валюта по настройкам ОС  
Console.WriteLine($"{price:E2}");    // 1,23E+003 — научная запись  
Console.WriteLine($"{price:P1}");    // 123 456,8 % — проценты (умножает на 100)  
Console.WriteLine($"{255:X}");       // FF — шестнадцатеричное (см. лекцию 3)  
Console.WriteLine($"{1234567:N0}");  // 1 234 567 — с разделителями разрядов  
  
// Выравнивание: {значение, ширина} — дополняет пробелами до ширины  
Console.WriteLine($"|{name,10}|");   // |      Анна| — вправо  
Console.WriteLine($"|{name,-10}|");  // |Анна      | — влево
```

5.5. Ввод с клавиатуры: Console.ReadLine

 `Console.ReadLine()` — метод, который **останавливает программу** и ждёт, пока пользователь наберёт текст и нажмёт *Enter*. Возвращает введённое как **строку** (`string`) — или `null`, если ввод кончился (*Ctrl+Z*).

Ключевое правило, на котором спотыкаются все новички:

 `ReadLine()` **всегда** возвращает строку. Даже если пользователь ввёл «25» — вы получите строку `"25"`, а не число. Складывать строки с числами нельзя! Нужен **преобразование (парсинг)** — см. ниже.

```
Console.Write("Как тебя зовут? ");           // Write, чтобы курсор остался на строке
string name = Console.ReadLine();           // программа ЖДЁТ ввода

Console.Write("Сколько тебе лет? ");
string ageText = Console.ReadLine();        // получили СТРОКУ, например "20"

// Console.WriteLine(ageText + 1);          // ОШИБКА ЛОГИКИ: "201" — склейка строк!
```

5.6. Превращаем строку в число: Parse, TryParse, Convert

Три инструмента превращения строки в число — и чёткое понимание, когда какой:

Способ 1. `int.Parse` — «преврати или урони»

```
string ageText = "20";
int age = int.Parse(ageText);           // строка → число, теперь можно считать:
Console.WriteLine(age + 1);             // 21 ✓

// ОПАСНО: если пользователь ввёл не число — программа УПАДЁТ с исключением:
// int.Parse("привет") → FormatException, аварийное завершение
```

Способ 2. `TryParse` — «попробуй и сообщи» (безопасный, РЕКОМЕНДУЕТСЯ)

 `TryParse` — попытка преобразования без исключений: возвращает `true` / `false` (получилось ли), а результат кладёт в переменную через `out`.

```

Console.Write("Введите возраст: ");
string input = Console.ReadLine();

bool success = int.TryParse(input, out int age);
//                                └─ сюда запишется число, ЕСЛИ получится

if (success)
{
    Console.WriteLine($"Через год будет {age + 1}");
}
else
{
    Console.WriteLine($"'{input}' — это не число. Попробуйте ещё раз.");
}

```

 **Разбор синтаксиса** `out` : слово `out` означает «метод запишет значение в эту переменную». Мы объявляем `int age` прямо в вызове — переменная создаётся там же.

 **Аналогия:** `Parse` — слепое доверие («прыгай — разберёмся»); `TryParse` — проверка парашюта перед прыжком. В программах, которые читают ввод пользователя, всегда используйте `TryParse`.

Способ 3. `Convert` — универсальный преобразователь

```

int a    = Convert.ToInt32("25");           // строка → int
double d = Convert.ToDouble("3.14");
bool b   = Convert.ToBoolean("true");

// ВАЖНО: Convert при неверном вводе ТОЖЕ бросает исключение — как Parse.

```

Шпаргалка: когда что использовать

Инструмент	Поведение при ошибке	Когда применять
<code>int.Parse(s)</code>	Исключение <code>FormatException</code>	Ввод гарантированно корректен (например, прочитали число из файла, который сами записали)

Инструмент	Поведение при ошибке	Когда применять
<code>int.TryParse(s, out x)</code>	Возвращает <code>false</code> , без исключений	Любой ввод пользователя — выбор по умолчанию
<code>Convert.ToInt32(s)</code>	Исключение	Легаси-код; удобно преобразовывать другие типы между собой

Запрос числа с повтором до успеха — шаблон для запоминания

```
int ReadPositiveInt(string prompt)           // своя функция: спрашивает число,
{                                           // пока пользователь не введёт корректное
    while (true)
    {
        Console.Write(prompt);
        string input = Console.ReadLine();

        if (int.TryParse(input, out int value) && value > 0)
            return value;                  // успех — возвращаем число

        Console.WriteLine("Нужно целое положительное число, попробуйте ещё раз.");
    }
}

int count = ReadPositiveInt("Сколько раз поздороваться? ");
```

💡 **Обратите внимание:** `&&` — «и»: обе части должны быть истинны. Если `TryParse` вернул `false`, вторая проверка даже не выполняется (короткое замыкание) — ошибки не будет.

5.7. Знакомство: `Console.ReadKey` и паузы

```
Console.WriteLine("Нажмите любую клавишу...");
Console.ReadKey();                // ждёт НАЖАТИЕ клавиши (не Enter!) — часто для паузы

ConsoleInfo info = default;      // ключ `true` — не отображать нажатую клавишу:
                                // Console.ReadKey(true)
```


Исправим на корректный пример:

```
Console.WriteLine("Нажмите любую клавишу для выхода...");  
Console.ReadKey(true);           // true = клавиша не будет напечатана на экране
```

5.8. Управление внешним видом консоли

```
Console.Title = "Моя программа";           // заголовок окна  
  
Console.ForegroundColor = ConsoleColor.Green; // цвет текста  
Console.WriteLine("Всё хорошо!");          // зелёное сообщение  
Console.ForegroundColor = ConsoleColor.Red;  // красное  
Console.WriteLine("Ошибка!");              // вернуть обычный цвет  
Console.ResetColor();  
  
Console.BackgroundColor = ConsoleColor.DarkBlue; // цвет фона текста  
Console.Write(" Выделение ");  
Console.ResetColor();  
  
Console.Clear();                           // очистить весь экран  
  
Console.SetCursorPosition(10, 5);          // переместить курсор: x=10, y=5  
Console.Write("Текст в заданной позиции"); // основа для «консольной графики»
```

Классический приём — цветные сообщения об ошибках:

```
void PrintColor(string text, ConsoleColor color)  
{  
    Console.ForegroundColor = color;  
    Console.WriteLine(text);  
    Console.ResetColor();  
}  
  
PrintColor("Успешно!", ConsoleColor.Green);  
PrintColor("Предупреждение", ConsoleColor.Yellow);  
PrintColor("Ошибка!", ConsoleColor.Red);
```

5.9. Аргументы командной строки: `string[] args`

Программе можно передать данные **при запуске** — ещё до того, как она что-то спросит:

```
dotnet run -- Анна 25
```

```
// Параметры args — массив строк из командной строки:
// (в top-level стиле args доступен сразу; в классическом — string[] args в Main)

if (args.Length == 0)
{
    Console.WriteLine("Использование: HelloApp <имя> <возраст>");
    return;    // завершаем программу
}

string name = args[0];                                // первый аргумент
Console.WriteLine($"Привет, {name}!");

if (args.Length > 1 && int.TryParse(args[1], out int age))
    Console.WriteLine($"Тебе {age} лет");
```

 **Где это встречается:** `git commit -m "текст"`, `dotnet new console -n Имя` — всё это аргументы командной строки. Умение их читать делает ваши утилиты профессиональными.

5.10. Сквозной пример: интерактивный калькулятор

Соберём всё изученное в одну работающую программу:

```
// ===== КОНСОЛЬНЫЙ КАЛЬКУЛЯТОР =====

Console.Title = "Калькулятор v1.0";
Console.WriteLine("=== Калькулятор ===");
Console.WriteLine("Введите два числа и операцию (+, -, *, /)");
Console.WriteLine("Для выхода введите 'q' вместо числа.\n");

while (true)    // главный цикл: работаем, пока пользователь не выйдет
{
    // --- ввод первого числа ---
    Console.Write("Первое число (или q): ");
    string input1 = Console.ReadLine();
    if (input1 == "q") break;                                // условие выхода

    if (!double.TryParse(input1, out double a))              // проверка корректности
```

```
{
    PrintError("Это не число. Повторите ввод.");
    continue; // пропустить итерацию
}

// --- ввод второго числа ---
Console.Write("Второе число: ");
if (!double.TryParse(Console.ReadLine(), out double b))
{
    PrintError("Это не число. Повторите ввод.");
    continue;
}

// --- ввод операции ---
Console.Write("Операция (+ - * /): ");
string op = Console.ReadLine();

// --- вычисление (switch-выражение, C# 8+) ---
double? result = op switch // double? = «число ИЛИ null»
{
    "+" => a + b,
    "-" => a - b,
    "*" => a * b,
    "/" => b != 0 ? a / b : null, // делить на ноль нельзя!
    _   => null                 // _ = всё остальное (неизвестная операция)
};

if (result is null)
{
    PrintError(b == 0 && op == "/"
        ? "Деление на ноль невозможно!"
        : "Неизвестная операция.");
}
else
{
    Console.ForegroundColor = ConsoleColor.Green;
    Console.WriteLine($"Результат: {a} {op} {b} = {result:F3}\n");
    Console.ResetColor();
}
}

Console.WriteLine("Спасибо за использование! Нажмите любую клавишу...");
Console.ReadKey(true);

// --- вспомогательная функция (локальная) ---
```

```
void PrintError(string message)
{
    Console.ForegroundColor = ConsoleColor.Red;
    Console.WriteLine("⚠ " + message + "\n");
    Console.ResetColor();
}
```

 Что здесь нового и почему работает:

- *while (true) ... break* — бесконечный цикл с выходом по команде: типовой каркас диалоговых программ;
- *continue* — «прервать текущий круг цикла и начать заново»;
- *double?* — «нулевые» значения: *null* означает «результата нет» (не число, а отсутствие);
- *switch* -выражение — компактный выбор по значению *or* ;
- локальная функция *PrintError* объявлена внизу, но доступна во всей программе.

5.11. Запуск и отладка в Visual Studio

Действие	Горячая клавиша	Что происходит
Запуск без отладки	Ctrl+F5	Программа выполняется, окно остаётся открытым после завершения
Запуск с отладкой	F5	Выполняется под управлением отладчика
Точка останова	F9 (на строке)	Программа остановится на этой строке — можно изучить значения переменных
Шаг с обходом	F10	Выполнить строку, не заходя внутрь методов
Шаг с заходом	F11	Выполнить строку, заходя внутрь вызванных методов
Смотреть значение	Навести курсор на переменную	Всплывающая подсказка со значением; окно Watch — для контроля

💡 **Главный навык новичка — отладчик, а не угадывание.** Не понимаете, почему программа работает не так? Поставьте точку останова, запустите F5 и смотрите, какие значения реально лежат в переменных. Это быстрее любых размышлений.

5.12. Публикация программы: делимся с другими

```
dotnet publish -c Release -r win-x64 --self-contained false -p:PublishSingleFile=true
```

Флаг	Значение
-c Release	Оптимизированная сборка (быстрее Debug)
-r win-x64	Целевая платформа (win-x64, linux-x64, osx-arm64...)
--self-contained true	Включить рантайм внутрь — работает на машине без установленного .NET (файл больше)
--self-contained false	Рантайм должен быть установлен (файл маленький)
- p:PublishSingleFile=true	Всё в один .exe

5.13. Выводы

1. Консольная программа создаётся за минуту: `dotnet new console + dotnet run` .
2. Ввод — **только строки** (`Console.ReadLine`), числа получаем через `TryParse` — это защищает от падений.
3. `WriteLine` + интерполяция `"${...}"` — стандарт вывода; форматы `F2` , `C` , `X` делают вывод красивым.
4. Каркас диалоговой программы: `while (true)` + условие выхода + проверка ввода + понятные сообщения об ошибках.
5. Отладчик (F5, F9, F10) — основной инструмент поиска ошибок.
6. `dotnet publish` превращает проект в распространяемую программу.

Практические задания:

1. **«Знакомство»:** спросите имя и год рождения, вычислите возраст (из текущего года) и выведите: «Привет, *имя*, тебе *N* лет». Защитите ввод через `TryParse` .

2. **«Конвертер»:** программа спрашивает температуру в °C и выводит в °F (формула $F = C \times 9/5 + 32$) и в Кельвинах ($K = C + 273,15$), с форматом `F1`.
3. **«Касса»:** пользователь вводит цену товара и количество (оба с проверкой); программа печатает итог с двумя знаками после запятой, скидку 10% при сумме > 5000 и цветной итог.
4. **Со звёздочкой — «Угадай число»:** программа загадывает случайное число (`Random rnd = new Random(); int secret = rnd.Next(1, 101);`), пользователь угадывает, программа подсказывает «больше»/«меньше» цветным текстом и считает попытки.
5. **Со звёздочкой — аргументы:** доработайте калькулятор: если при запуске переданы три аргумента (число, операция, число) — считать сразу без вопросов: `dotnet run -- 5 * 3`.

Лекция 6. Основы программирования на C# : переменные, типы, операторы, условия, циклы, массивы, методы и исключения

Словарик к лекции

Термин	Что означает
Переменная	Именованная ячейка памяти определённого типа для хранения значения
Литерал	Значение, записанное прямо в коде: <code>42</code> , <code>3.14</code> , <code>"текст"</code> , <code>true</code>
Выражение	Конструкция из значений, переменных и операторов, дающая результат
Условие	Логическое выражение (<code>true/false</code>), управляющее ветвлением программы
Итерация	Один проход (круг) цикла
Массив	Набор элементов одного типа, хранящихся подряд, с доступом по индексу
Индекс	Номер элемента в массиве; нумерация с нуля!

Термин	Что означает
Сигнатура метода	Имя метода + список типов параметров (то, по чему метод опознаётся)
Область видимости	Часть кода, где переменная существует и доступна
Исключение	Объект-ошибка, выбрасываемый при невозможности продолжить работу; можно перехватить

6.1. Переменные: ячейки для данных

 **Переменная** — именованная область памяти с указанным типом. Синтаксис объявления: `Тип имя = значение;`

 **Аналогия:** переменная — **почтовый ящик с табличкой**. Тип — размер ящика (в него «большое» не влезет), имя — надпись на табличке, значение — содержимое. Содержимое можно менять, но только на подходящее по размеру (типу).

```
int age = 20;           // объявили и сразу инициализировали
double price;           // можно объявить без значения — но ИСПОЛЬЗОВАТЬ
price = 99.90;          // нельзя, пока не присвоим!

age = 21;               // меняем значение (тип тот же)

const double TaxRate = 0.20; // КОНСТАНТА: менять нельзя, компилятор проследит
// TaxRate = 0.25;         // ОШИБКА КОМПИЛЯЦИИ

var count = 10;          // var: компилятор SAM выводит тип по правой части.
var name = "Анна";        // count — это int, name — это string.
                           // var — не «переменная без типа»! Тип всё равно строгий.

// Правила имён: camelCase для переменных (userName),
// PascalCase для классов и методов (GetUserName),
// имя должно объяснять СМЫСЛ: customerCount лучше, чем x2.
```

Область видимости — где переменная живёт:

```
{
    int local = 5;           // переменная существует только ВНУТРИ этих фигурных скобок
    Console.WriteLine(local); // ✓ здесь доступна
}
// Console.WriteLine(local); // X ОШИБКА: local здесь уже не существует
```

 **Правило:** переменная живёт от строки объявления до закрывающей скобки блока, где объявлена. Ошибка «the name does not exist» почти всегда значит «вы вышли за границы области видимости» или «объявили не там».

6.2. Типы данных: фундамент

6.2.1. Обзор базовых типов

Категория	Типы	Пример литерала
Целые числа	int , long , short , byte	42 , 1_000_000
Дробные	double , float , decimal	3.14 , 0.5m
Логический	bool	true , false
Символ	char	'А' (одинарные кавычки!)
Текст	string	"Привет" (двойные кавычки)
«Пустое» значение	null	только для ссылочных типов

6.2.2. Выбор типа на практике

```
int visitors = 1250;           // счётчики, количества, ID → int
long population = 8_100_000_000L; // большие числа → long (L – суффикс)
double temperature = 36.6;      // физика, координаты, средние → double
decimal account = 1500.50m;     // ДЕНЬГИ → decimal (m – суффикс)
bool isVip = true;              // да/нет → bool
char grade = 'A';               // ОДИН символ → char
string fullName = "Иванов Анна"; // текст → string
```

 **Три частые ошибки с типами:**

1. Деньги в `double` → копейки теряются (см. лекцию 3). Только `decimal` !
2. `'A'` и `"A"` — разные вещи: `char` и `string`. Одинарные/двойные кавычки — не взаимозаменяемы.
3. $10 / 3 = 3$, а не 3,33: если оба операнда целые, деление целочисленное (дробная часть отбрасывается).

6.2.3. Преобразования типов

Неявное (безопасное, без потерь — «маленькое» в «большое»):

```
int i = 100;
long big = i;           // int → long: помещается всегда, компилятор разрешает
double d = i;           // int → double: тоже безопасно
```

Явное (возможна потеря — требуется ручное указание):

```
double pi = 3.99;
int cut = (int)pi;       // ЯВНОЕ: дробная часть ОТБРОШЕНА (не округлена!)
Console.WriteLine(cut);  // 3 — не 4!

// Для округления есть математика:
Console.WriteLine((int)Math.Round(pi)); // 4 — математическое округление
Console.WriteLine(Math.Ceiling(pi));    // 4 — вверх
Console.WriteLine(Math.Floor(pi));      // 3 — вниз
Console.WriteLine(Math.Truncate(pi));   // 3 — просто отбросить дробную часть
```

Строка ↔ число — уже знакомо по лекции 5: `int.TryParse`, `int.Parse`, `ToString()`.

```
int n = 42;
string text = n.ToString();           // "42" — число в строку (всегда безопасно)
```

6.3. Операторы: действия над данными

6.3.1. Арифметические

Оператор	Смысл	Пример	Результат
<code>+</code> <code>-</code> <code>*</code>	обычные	<code>7 * 3</code>	21

Оператор	Смысл	Пример	Результат
/	деление	7 / 2	3 (целочисленное!)
/	деление	7 / 2.0	3.5 (есть double — деление дробное)
%	остаток от деления	7 % 2	1
++	+1	i++	
--	-1	i--	

 **Остаток % — недооценённый герой.** Классические применения:

```
Console.WriteLine(17 % 2); // 1 → число нечётное (0 было бы чётным)
Console.WriteLine(100 % 7); // 2 → сколько останется, разложив по 7
Console.WriteLine(125 % 10); // 5 → последняя цифра числа
Console.WriteLine(minute % 60); // минуты в пределах часа (цикличность)
```

Тонкость ++ — до или после:

```
int i = 5;
Console.WriteLine(i++); // 5 — СНАЧАЛА использовать, потом увеличить
Console.WriteLine(i);   // 6
```

```
int j = 5;
Console.WriteLine(++j); // 6 — СНАЧАЛА увеличить, потом использовать
```

 **Совет:** на старте пишите `i++` отдельной строкой — не смешивайте с выводом, меньше путаницы.

6.3.2. Сравнения — результат всегда `bool`

```
int a = 5, b = 7;
Console.WriteLine(a == b); // False — равенство (ДВА знака ==)
Console.WriteLine(a != b); // True  — не равно
Console.WriteLine(a < b);  // True
Console.WriteLine(a >= b); // False
```

⚠ **Ошибка №1 всех новичков:** `=` — присваивание, `==` — сравнение. `if (x = 5)` не скомпилируется для чисел (хорошо!), но с `bool` пройдёт — будьте внимательны.

6.3.3. Логические операторы

Оператор	Название	Истинно, когда
<code>&&</code>	И (AND)	оба условия истинны
<code> </code>	ИЛИ (OR)	хотя бы одно истинно
<code>!</code>	НЕ (NOT)	инвертирует: <code>!true</code> → <code>false</code>

```
int age = 25;
bool hasTicket = true;
```

```
Console.WriteLine(age >= 18 && hasTicket); // True — и возраст, и билет
Console.WriteLine(age < 18 || hasTicket);  // True — достаточно одного
Console.WriteLine(!hasTicket);             // False — «нет билета»
```

```
// КОРОТКОЕ ЗАМЫКАНИЕ: если первый операнд && ложен, второй НЕ вычисляется:
string s = null;
// if (s.Length > 0 && s != null) — ПЛОХО: упадёт на s.Length при null
if (s != null && s.Length > 0) // ХОРОШО: при null вторая часть не выполнится
    Console.WriteLine(s);
```

6.3.4. Приоритет операторов (кратко)

Как в математике: `*` / `%` выше, чем `+` - ; сравнения выше, чем `&&` ; `&&` выше, чем `||` .

Скобки — всегда можно и нужно:

```
Console.WriteLine(2 + 3 * 4); // 14 — сначала умножение
Console.WriteLine((2 + 3) * 4); // 20 — скобки меняют порядок
Console.WriteLine(age >= 18 && (hasTicket || isGuest));
```

💡 **Правило стиля:** если сомневаетесь в приоритете — ставьте скобки. Читаемость важнее краткости.

6.4. Условия: программа принимает решения

6.4.1. if / else if / else

```
Console.Write("Введите температуру: ");
double t = double.Parse(Console.ReadLine());

if (t <= 0)
{
    Console.WriteLine("Лёд");
}
else if (t < 100)
{
    Console.WriteLine("Вода");
}
else
{
    Console.WriteLine("Пар");
}
```

Правила конструкции:

- проверка идёт **сверху вниз**; выполняется **первая** подошедшая ветка, остальные пропускаются;
- `else` — необязательная «лови всё остальное»;
- фигурные скобки `{ }` можно опустить для одной команды, но **новичкам рекомендуем всегда писать** — меньше ошибок.

6.4.2. Вложенные условия и логика вместо вложенности

```
// ВЛОЖЕННОСТЬ — работает, но читается тяжело:
if (age >= 18)
{
    if (hasPassport)
    {
        Console.WriteLine("Вход разрешён");
    }
}
```

```
// ТО ЖЕ САМОЕ через && — компактнее и яснее:
if (age >= 18 && hasPassport)
    Console.WriteLine("Вход разрешён");
```

6.4.3. Тернарный оператор: if в одну строку

```
// условие ? значение_если_да : значение_если_нет
int a = 10, b = 20;
int max = a > b ? a : b;           // 20

Console.WriteLine($"Статус: {(age >= 18 ? "взрослый" : "ребёнок")}");
```

 Используйте для **простого выбора значения**. Вложенные тернарники ($a ? b ? c : d : e$) — путь к нечитаемому коду.

6.4.4. switch: выбор одного из вариантов

Когда одна переменная сравнивается с набором значений — `switch` нагляднее цепочки `else if`:

```
Console.Write("Введите оценку (2-5): ");
int mark = int.Parse(Console.ReadLine());

switch (mark)
{
    case 5:
        Console.WriteLine("Отлично!");
        break;           // break ОБЯЗАТЕЛЕН — конец ветки
    case 4:
        Console.WriteLine("Хорошо");
        break;
    case 3:
        Console.WriteLine("Удовлетворительно");
        break;
    case 2:
        Console.WriteLine("Неудовлетворительно");
        break;
    default:
        // «всё остальное»
        Console.WriteLine("Нет такой оценки");
        break;
}
```

Несколько значений в одной ветке и switch-выражение (современный стиль):

```
// Классический: несколько case подряд:
string dayType;
```



```

Console.Write($"{i} ");           // 10 9 8 ... 1

// Таблица умножения — ВЛОЖЕННЫЕ циклы:
for (int row = 1; row <= 9; row++)
{
    for (int col = 1; col <= 9; col++)
        Console.Write($"{row * col,4}"); // выравнивание ширины 4
    Console.WriteLine();                 // перевод строки после ряда
}

```

Разбор анатомии for:

```

for (инициализация; условие; шаг)
{
    тело;
}

```

Порядок работы: ① инициализация (один раз) → ② проверка условия → ③ тело → ④ шаг → ② проверка → ③ тело... пока условие не станет ложным.

6.5.2. while — когда повторяем ПОКА условие истинно

```

int sum = 0;
int number = 1;

while (sum <= 100)           // «пока сумма не перевалит за 100»
{
    sum += number;           // += это sum = sum + number
    number++;
}

Console.WriteLine($"Достаточно сложить числа от 1 до {number - 1}"); // 14

```

⚠ Бесконечный цикл — классическая ошибка: забыли изменить переменную условия.

```

// int i = 0;
// while (i < 10)
//     Console.WriteLine(i); // i никогда не растёт → цикл ВЕЧНЫЙ (Ctrl+C!)

```

6.5.3. do-while — выполнит тело ХОТЯ БЫ РАЗ

```

string password;

```

```
do
{
    Console.Write("Введите пароль: ");
    password = Console.ReadLine();
} while (password != "qwerty");    // проверка ПОСЛЕ тела → тело точно исполнится 1 раз

Console.WriteLine("Доступ разрешён!");
```

 Как выбрать цикл:

Ситуация	Цикл
Знаю, сколько раз повторить	<i>for</i>
Повторять, пока условие верно; может не выполниться ни разу	<i>while</i>
Сначала спросить, потом проверять (меню, пароль)	<i>do-while</i>
Перебрать элементы коллекции	<i>foreach</i> (чуть ниже)

6.5.4. break и continue — управление циклом

```
// break — немедленно ВЫЙТИ из цикла:
for (int i = 1; i <= 100; i++)
{
    if (i * i > 500)
    {
        Console.WriteLine($"Первый квадрат больше 500: {i}²={i * i}");
        break;                // дальше проверять бессмысленно
    }
}

// continue — ПРОПУСТИТЬ остаток итерации, перейти к следующей:
for (int i = 1; i <= 10; i++)
{
    if (i % 2 != 0)
        continue;            // нечётные пропускаем
    Console.Write($"{i} ");    // 2 4 6 8 10
}
```


6.5.5. foreach — элегантный перебор коллекций

```
string[] fruits = { "яблоко", "банан", "вишня" };

foreach (string fruit in fruits)    // «для каждого элемента...»
{
    Console.WriteLine(fruit);      // индексы не нужны — просто элементы!
}
```

 *В foreach нельзя изменять коллекцию (добавлять/удалять элементы) — только читать.*

6.6. Массивы: много значений под одним именем

 **Массив** — фиксированный набор элементов **одного типа**, хранящихся в памяти подряд, с доступом по **индексу**.

 **Железное правило: индексация начинается с 0.** Первый элемент — `[0]`, последний — `[длина - 1]`. Обращение за границы → исключение `IndexOutOfRangeException`.

```
// — Создание —————
int[] scores = new int[5];           // массив из 5 нулей (значения по умолчанию)
int[] primes = { 2, 3, 5, 7, 11 };   // сразу с значениями — размер вычислится
string[] names = new string[3];     // массив из 3 null'ов

// — Доступ по индексу —————
Console.WriteLine(primes[0]);        // 2 — ПЕРВЫЙ элемент
Console.WriteLine(primes[4]);        // 11 — ПОСЛЕДНИЙ (5 - 1)
Console.WriteLine(primes.Length);    // 5 — длина (без скобок!)

primes[2] = 99;                      // запись: заменяем 5 на 99
// Console.WriteLine(primes[5]);     // X ПАДЕНИЕ: индексов 0..4, пять — вне границ

// — Перебор —————
for (int i = 0; i < primes.Length; i++)    // классика: нужен индекс
    Console.Write($"{primes[i]} ");

foreach (int p in primes)                // лаконично: нужен только элемент
```

```
Console.Write($"{p} ");

// — Полезные операции —————
int[] data = { 8, 3, 15, 6, 1 };

int sum = 0;
foreach (int x in data) sum += x;
Console.WriteLine(sum);                // 33

Console.WriteLine(data.Max());          // 15 — максимум (System.Linq)
Console.WriteLine(data.Min());          // 1
Console.WriteLine(data.Average());      // 6,6
Array.Sort(data);                       // сортировка на месте: 1 3 6 8 15
Array.Reverse(data);                    // разворот
```

Классический алгоритм — поиск максимума вручную (важно понимать механику, не только `Max()`):

```
int[] numbers = { 8, 3, 15, 6, 1 };

int max = numbers[0];                  // начинаем с ПЕРВОГО элемента
foreach (int x in numbers)
{
    if (x > max)
        max = x;                      // нашли больше — обновляем
}
Console.WriteLine(max);                // 15
```

Массив как буфер ввода — типовая задача:

```
Console.Write("Сколько оценок ввести? ");
int n = int.Parse(Console.ReadLine());

int[] grades = new int[n];             // массив нужного размера

for (int i = 0; i < n; i++)
{
    Console.Write($"Оценка #{i + 1}: "); // человеку удобнее с 1
    grades[i] = int.Parse(Console.ReadLine());
}

Console.WriteLine($"Средний балл: {grades.Average():F2}");
```

6.7. Методы: разбиваем программу на части

 **Метод** — именованный блок кода, который можно вызывать многократно.

Синтаксис: `ВозвращаемыйТип Имя(параметры) { тело }`

 **Зачем:** метод — это **глагол с именем**. Вместо того чтобы каждый раз писать «возьми список, пройди по нему, сравни с текущим максимумом...», вы пишете `FindMax(numbers)` — код читается как предложение.

6.7.1. Первые методы

```
// void = метод НИЧЕГО не возвращает, просто выполняет действия
void PrintLine()
{
    Console.WriteLine("=====");
}

void GreetUser(string name)           // параметр — «входные данные» метода
{
    Console.WriteLine($"Привет, {name}!");
}

// Методы с ВЕРНУТЫМ значением:
int Square(int x)
{
    return x * x;                     // return = «вернуть результат и выйти»
}

double GetAverage(int a, int b)
{
    return (a + b) / 2.0;              // 2.0 — чтобы деление было дробным!
}

// — Вызовы —
PrintLine();                          // скобки ОБЯЗАТЕЛЬНЫ даже без параметров
GreetUser("Анна");                     // "Анна" — аргумент
int result = Square(7);                 // результат СНАЧАЛА возвращается,
Console.WriteLine(result);              // ПОТОМ сохраняется/используется: 49
Console.WriteLine(GetAverage(3, 8));    // 5.5
```

⚠ Метод, объявленный с типом `int`, **обязан** вернуть число по всем путям выполнения. Забыли `return` → ошибка компиляции.

6.7.2. Параметры по умолчанию и именованные аргументы

```
void Log(string message, string level = "INFO")    // level имеет значение по умолчанию
{
    Console.WriteLine($"[{level}] {message}");
}

Log("Приложение запущено");                      // [INFO] Приложение запущено
Log("Файл не найден", "ERROR");                  // [ERROR] Файл не найден
Log(level: "WARN", message: "Мало памяти");      // именованные аргументы — порядок свободный
```

6.7.3. out и ref: параметры-«возвраты»

```
// out: метод ЗАПИСЫВАЕТ значение в параметр (второй «выход» метода)
// (вы уже видели: int.TryParse(input, out int value))

bool Divide(int a, int b, out double result)
{
    if (b == 0)
    {
        result = 0;                // out-параметру НАДО присвоить значение ЛЮБОМУ пути
        return false;
    }
    result = (double)a / b;
    return true;
}

if (Divide(10, 3, out double res))
    Console.WriteLine($"10 / 3 = {res:F3}");
else
    Console.WriteLine("Делить на ноль нельзя");
```

6.7.4. Перегрузка: одно имя — разные параметры

```
void Print(int value)    => Console.WriteLine($"int: {value}");
void Print(double value) => Console.WriteLine($"double: {value}");
```

```
void Print(string value)    => Console.WriteLine($"string: {value}");

Print(42);                  // компилятор выберет ПО ТИПАМ аргументов
Print(3.14);
Print("текст");
```

6.7.5. Пример рефакторинга: «до» и «после»

```
// ❌ ПЛОХО: вся логика в одной куче, повторяющийся код
Console.Write("Введите радиус: ");
double r1 = double.Parse(Console.ReadLine());
Console.WriteLine($"Площадь: {Math.PI * r1 * r1:F2}");

Console.Write("Введите радиус: ");
double r2 = double.Parse(Console.ReadLine());
Console.WriteLine($"Длина окружности: {2 * Math.PI * r2:F2}");

// ✓ ХОРОШО: каждый метод отвечает за одно, повторы исчезли
double ReadPositiveDouble(string prompt)
{
    while (true)
    {
        Console.Write(prompt);
        if (double.TryParse(Console.ReadLine(), out double value) && value > 0)
            return value;
        Console.WriteLine("Нужно положительное число!");
    }
}

double CircleArea(double r)    => Math.PI * r * r;
double CircleCircumference(double r) => 2 * Math.PI * r;

double r = ReadPositiveDouble("Введите радиус: ");
Console.WriteLine($"Площадь: {CircleArea(r):F2}");
Console.WriteLine($"Длина окружности: {CircleCircumference(r):F2}");
```

6.8. Строки: методы, которыми пользуются каждый день

```
string text = " Hello, C# World! ";
```

```
// — Основные свойства и методы —
Console.WriteLine(text.Length);           // 19 — длина
Console.WriteLine(text.Trim());           // "Hello, C# World!" — убрать пробелы
Console.WriteLine(text.ToUpper());        // ВЕРХНИЙ регистр
Console.WriteLine(text.ToLower());        // нижний регистр
Console.WriteLine(text.Contains("C#"));   // True — содержит ли подстроку
Console.WriteLine(text.StartsWith(" He")); // True
Console.WriteLine(text.Replace("World", "Мир")); // замена
Console.WriteLine(text.Substring(2, 5));   // "Hello" — срез с 2-й позиции, 5 символа

// — Разделение и склейка —
string csv = "Анна;25;Москва";
string[] parts = csv.Split(';');          // массив из 3 частей
Console.WriteLine(parts[0]);              // Анна

string joined = string.Join(" | ", parts); // Анна | 25 | Москва

// — Сравнение БЕЗ учёта регистра (частая задача) —
Console.WriteLine("да" == "Да");          // False!
Console.WriteLine(string.Equals("да", "Да",
    StringComparison.OrdinalIgnoreCase));   // True

// — Проверки —
string input = " ";
Console.WriteLine(string.IsNullOrWhiteSpace(input)); // True — пусто или одни пробелы

// — Доступ к символам —
string word = "Привет";
Console.WriteLine(word[0]);               // П — первый символ (char!)
foreach (char c in word)
    Console.Write($"{c} ");              // П р и в е т
```

⚠ Строки неизменяемы: методы `ToUpper`, `Replace`, `Trim` **не меняют** исходную строку — они **возвращают новую**. Классическая ошибка: вызвать `text.Trim()`; и ожидать, что `text` изменился.

```
string s = "  текст  ";
s = s.Trim();           // ПРАВИЛЬНО: результат присваиваем обратно
```

6.9. Коллекция List<T>: «резиновый» массив

Массив имеет фиксированный размер. Когда количество элементов заранее неизвестно — используйте List<T> :

```
List<string> tasks = new List<string>();           // пустой список строк

tasks.Add("Сделать уроки");                      // добавить в конец
tasks.Add("Выучить C#");
tasks.Add("Поспать");

Console.WriteLine(tasks.Count);                  // 3 (у массива было Length!)
Console.WriteLine(tasks[1]);                     // Выучить C# — доступ по индексу КАК у м

tasks.Remove("Поспать");                         // удалить по значению
tasks.RemoveAt(0);                              // удалить по индексу

tasks.Insert(0, "Позавтракать");                 // вставить на позицию

foreach (string task in tasks)                  // перебор
    Console.WriteLine($"□ {task}");

Console.WriteLine(tasks.Contains("Выучить C#")); // True — есть ли элемент
```

Список чисел с ввода — типовой паттерн:

```
List<int> numbers = new List<int>();

Console.WriteLine("Введите числа (пустая строка = конец):");
while (true)
{
    string input = Console.ReadLine();
    if (string.IsNullOrEmpty(input))
        break;                                     // пустой ввод — стоп

    if (int.TryParse(input, out int n))
        numbers.Add(n);
    else
        Console.WriteLine("Это не число, пропускаю.");
}
```

```
if (numbers.Count > 0)
    Console.WriteLine($"Введено {numbers.Count} чисел; сумма = {numbers.Sum()}");
```

6.10. Исключения: программа не падает, а обрабатывает ошибки

 **Исключение (exception)** — объект, сигнализирующий об ошибке времени выполнения. Возникнув, оно «всплывает» вверх по коду; если его никто не перехватил — программа аварийно завершается. Перехват — конструкцией `try/catch`.

```
Console.Write("Введите число: ");
string input = Console.ReadLine();

try
{
    int n = int.Parse(input);           // здесь МОЖЕТ возникнуть FormatException
    Console.WriteLine($"100 / n = {100 / n}"); // здесь МОЖЕТ возникнуть DivideByZeroEx
}
catch (FormatException)                // перехватываем КОНКРЕТНЫЙ тип
{
    Console.WriteLine($"'{input}' — не целое число.");
}
catch (DivideByZeroException)
{
    Console.WriteLine("На ноль делить нельзя!");
}
catch (Exception ex)                  // ЛОВУШКА ВСЕГО — всегда последняя!
{
    Console.WriteLine($"Неожиданная ошибка: {ex.Message}");
}
finally
{
    Console.WriteLine("Блок finally: выполняется ВСЕГДА — с ошибкой или без.");
}
```

Разбор конструкции:

Блок	Назначение
<code>try { }</code>	«Подозрительный» код, который может упасть
<code>catch (Тип) { }</code>	План действий при конкретной ошибке. Блоков может быть несколько — выбирается первый подходящий по типу
<code>catch (Exception ex)</code>	Универсальный перехват; <code>ex.Message</code> — текст ошибки, <code>ex.StackTrace</code> — где произошла. Ставить только последним
<code>finally { }</code>	Выполняется в любом случае (была ошибка или нет) — для освобождения ресурсов (закреть файл, соединение)

Порядок catch важен: от частных типов к общему `Exception` , иначе общий блок «съест» всё раньше частных.

Взаимосвязь TryParse и try/catch: для **ожидаемых** пользовательских ошибок — `TryParse` (быстро, без «исключений как потока управления»); `try/catch` — для неожиданных ситуаций (файл недоступен, сеть отвалилась).

6.11. Мини-проект: «Менеджер задач» (консольный)

Финальный пример, связывающий **все** темы лекции. Изучите его построчно — здесь каждый элемент из разделов 6.1–6.10:

```
// ===== КОНСОЛЬНЫЙ МЕНЕДЖЕР ЗАДАЧ =====
// Демонстрирует: списки, циклы, условия, методы, switch, TryParse, строки

List<(string Title, bool Done)> tasks = new(); // список кортежей: текст + статус

while (true) // главный цикл программы
{
    ShowMenu();
    string choice = Console.ReadLine();

    switch (choice)
    {
        case "1": AddTask(); break;
        case "2": ShowTasks(); break;
        case "3": CompleteTask(); break;
        case "4": RemoveTask(); break;
        case "0":
```

```
        Console.WriteLine("До встречи!");
        return; // выход из программы
    default:
        Print("Нет такой команды.", ConsoleColor.Red);
        break;
    }
    Console.WriteLine();
}

// ----- методы -----

void ShowMenu()
{
    Print("== МЕНЮ ЗАДАЧ ==", ConsoleColor.Cyan);
    Console.WriteLine("1 – добавить задачу");
    Console.WriteLine("2 – показать все");
    Console.WriteLine("3 – отметить выполненной");
    Console.WriteLine("4 – удалить");
    Console.WriteLine("0 – выход");
    Console.Write("Ваш выбор: ");
}

void AddTask()
{
    Console.Write("Текст задачи: ");
    string title = Console.ReadLine()?.Trim(); // ?. – вызвать Trim, если не null

    if (string.IsNullOrEmpty(title))
    {
        Print("Пустая задача – не добавляю.", ConsoleColor.Yellow);
        return;
    }
    tasks.Add((title, Done: false));
    Print("Задача добавлена!", ConsoleColor.Green);
}

void ShowTasks()
{
    if (tasks.Count == 0)
    {
        Print("Список пуст.", ConsoleColor.Yellow);
        return;
    }

    for (int i = 0; i < tasks.Count; i++)
```

```
{
    var (title, done) = tasks[i];                // распаковка кортежа
    string icon = done ? "☑" : "☐";
    var color = done ? ConsoleColor.DarkGray : ConsoleColor.White;

    Console.ForegroundColor = color;
    Console.WriteLine($"{i + 1,2}. {icon} {title}"); // выравнивание номера
    Console.ResetColor();
}
}

void CompleteTask()
{
    if (!TryReadTaskNumber("Номер выполненной задачи: ", out int index)) return;

    tasks[index] = (tasks[index].Title, Done: true);
    Print("Готово! ☑", ConsoleColor.Green);
}

void RemoveTask()
{
    if (!TryReadTaskNumber("Номер удаляемой задачи: ", out int index)) return;

    tasks.RemoveAt(index);
    Print("Задача удалена.", ConsoleColor.Green);
}

// Общий помощник: безопасно спрашивает номер задачи (от 1 до Count)
// и переводит его в индекс (от 0 до Count-1)
bool TryReadTaskNumber(string prompt, out int index)
{
    index = -1;
    if (tasks.Count == 0)
    {
        Print("Список пуст.", ConsoleColor.Yellow);
        return false;
    }

    Console.Write(prompt);
    if (!int.TryParse(Console.ReadLine(), out int n) ||
        n < 1 || n > tasks.Count)
    {
        Print($"Нужно число от 1 до {tasks.Count}.", ConsoleColor.Red);
        return false;
    }
}
```

```

    index = n - 1;           // человек считает с 1, массив — с 0!
    return true;
}

void Print(string text, ConsoleColor color)
{
    Console.ForegroundColor = color;
    Console.WriteLine(text);
    Console.ResetColor();
}

```

☞ Что здесь продемонстрировано:

- `while (true) + switch` — каркас меню-программы (лекция 5 + 6.4.4);
- `List<(string, bool)>` с кортежами — простая структура данных без создания класса;
- `TryReadTaskNumber` — **общий** помощник вместо дублирования проверки в двух методах (принцип DRY — *Don't Repeat Yourself*);
- везде `TryParse` и проверки границ — программа **не падает** ни при каком вводе;
- локальные методы — логика разбита по смыслу, главный цикл читается как оглавление.

6.12. Выводы

1. **Переменные** строго типизированы; имя должно объяснять смысл; `const` — для неизменных значений.
2. **Типы:** `int` — целые, `double` — дробные расчёты, `decimal` — деньги, `bool` — логика, `string` — текст. Помните: `10/3 = 3`; `'A' ≠ "A"`; `= ≠ ==`.
3. **Условия:** `if/else` — диапазоны и логика; `switch` — выбор по значениям; тернарник — выбор значения в одну строку.
4. **Циклы:** `for` — известно число повторов, `while` — «пока условие», `do-while` — «хотя бы раз», `foreach` — перебор коллекций. `break` — выйти, `continue` — пропустить итерацию.
5. **Массивы** — фиксированный размер, индексация с 0, длина `.Length`; **List<T>** — растёт динамически, длина `.Count`.
6. **Методы** — основа читаемого кода: одна задача — один метод; `TryParse`-стиль (`bool + out`) — образец безопасного API.
7. **Строки неизменяемы** — методы возвращают новые строки; присваивайте результат обратно.

8. **Исключения:** `try/catch/finally` ; для ожидаемого пользовательского ввода — `TryParse` , а не исключения.

Контрольные вопросы:

1. Чем `int` отличается от `double` при делении `7 / 2` ? Как получить 3,5?
2. Почему `10 / 3` даёт 3, а `10 / 3.0` — 3,33?
3. Когда выбрать `while` , а когда `do-while` ? Приведите по примеру.
4. Что произойдёт при обращении к `arr[10]` в массиве из 10 элементов? Какой тип исключения?
5. Почему `text.Trim()` не меняет строку `text` ? Как записать правильно?
6. Чем `foreach` удобнее `for` ? Когда `for` незаменим?
7. Зачем параметр `out` ? Напишите сигнатуру метода, возвращающего одновременно частное и остаток.
8. В каком порядке располагать блоки `catch` и почему?
9. Почему в менеджере задач проверка номера вынесена в отдельный метод `TryReadTaskNumber` ? Какой принцип это иллюстрирует?

Практические задания (по возрастанию сложности):

1. **FizzBuzz** (классика собеседований): числа от 1 до 100; вместо кратных 3 — «Fizz», кратных 5 — «Buzz», кратных 15 — «FizzBuzz».
2. **Реверс:** введите 5 чисел в массив, выведите в обратном порядке (без `Reverse` — циклом от конца к началу).
3. **Анализ числа:** введите число; выведите сумму его цифр (подсказка: `% 10` и `/ 10` в цикле), количество цифр и является ли оно палиндромом.
4. **Меню конвертера:** программа с циклом-меню: 1 — °C→°F, 2 — км→мили, 3 — фунты→кг, 0 — выход. Каждый пункт — отдельный метод.
5. **Калькулятор v2:** доработайте калькулятор из лекции 5 — добавьте степень (`Math.Pow`) и корень (`Math.Sqrt`); оформление вынесите в метод `PrintResult` .
6. **Менеджер задач v2:** добавьте пункты «показать только невыполненные», «очистить все выполненные», «редактировать текст задачи». Используйте LINQ -минимум: `Count(t => !t.Done)` .
7. *Со звёздочкой:* угадай-число с подсказками и защитой от нечислового ввода; после победы покажите статистику: попыток, среднее время (подсказка: `DateTime.Now`).

Рекомендуемые ресурсы к лекциям 5–6

1. **Официальные обучающие материалы Microsoft** — learn.microsoft.com/dotnet/csharp (разделы *Intro to C# tutorials, fundamentals*).
2. Э. Троелсен, Ф. Джепикс — «С# 10 и платформа .NET 6» (главы о языке).
3. metanit.com/sharp — разделы «Основы С#», «Массивы», «Методы» — с задачами для тренировки.
4. **Практика задачами:** codewars.com (уровень 8 kyu → 7 kyu — точно соответствует материалу этих лекций).
5. Отладчик вашей IDE — лучший «учебник»: ставьте точки останова и наблюдайте, как меняются переменные на каждом шаге цикла.

💡 **Как закреплять материал:** после каждой темы решайте хотя бы одну задачу, набирая код **вручную** (не копирующей). Мышечная память и ошибки, которые вы сами исправите, — самый быстрый путь к уверенности.